

# Deep Reinforcement Learning for Multi-Asset Trading

Rousomanis Georgios (10703)

May 2026

# Assignment 1: Deep Reinforcement Learning for Trading

## Motivation

- Financial markets are sequential, stochastic, and non-stationary
- Traditional trading strategies struggle under regime shifts
- Deep Reinforcement Learning enables adaptive decision-making directly from market data

## Developed Framework

### End-to-End Trading Pipeline

- Custom multi-asset trading environments
- Deep Reinforcement Learning agents
- Temporal feature extractors (MLP, LSTM, Transformer)
- Fuzzy Logic-based risk management
- Genetic Algorithm hyperparameter optimization

Goal: Develop and evaluate an intelligent trading framework capable of learning profitable and risk-aware portfolio management policies.

# Trading Environment

- Multi-asset portfolio (stocks + crypto)
- Gymnasium-based sequential decision process
- Observation contains:
  - Portfolio state (cash + holdings)
  - Market state (prices + indicators)
  - Risk signals (VIX, turbulence for stocks)

# Trading Environment

- Multi-asset portfolio (stocks + crypto)
- Gymnasium-based sequential decision process
- Observation contains:
  - Portfolio state (cash + holdings)
  - Market state (prices + indicators)
  - Risk signals (VIX, turbulence for stocks)

State representation:

$$s_t = [s_t^{portfolio}, s_t^{market}]$$

# Trading Environment

- Multi-asset portfolio (stocks + crypto)
- Gymnasium-based sequential decision process
- Observation contains:
  - Portfolio state (cash + holdings)
  - Market state (prices + indicators)
  - Risk signals (VIX, turbulence for stocks)

State representation:

$$s_t = [s_t^{portfolio}, s_t^{market}]$$

Continuous action space:

$$a_t \in [-1, 1]^N$$

# Trading Environment

- Multi-asset portfolio (stocks + crypto)
- Gymnasium-based sequential decision process
- Observation contains:
  - Portfolio state (cash + holdings)
  - Market state (prices + indicators)
  - Risk signals (VIX, turbulence for stocks)

State representation:

$$s_t = [s_t^{portfolio}, s_t^{market}]$$

Continuous action space:

$$a_t \in [-1, 1]^N$$

Reward:

$$r_t = \log \left( \frac{V_{t+1} + \epsilon}{V_t + \epsilon} \right) - \lambda e_t v_t$$

- $e_t$ : portfolio exposure
- $v_t$ : market volatility
- $\lambda$ : risk sensitivity coefficient

# Policy Architectures

- MLP (baseline)
- LSTM (temporal memory)
- Transformer (attention-based modeling)
- Hybrid MLP–Transformer

## Key Challenge

Financial data requires modeling both:

- temporal dependencies
- cross-asset interactions

# Transformer Feature Extractor

## Input Construction

$$X = [x_{t-T+1}, \dots, x_t]$$

where each observation contains both portfolio and market information:

$$x_t = [s_t^{\text{portfolio}}, s_t^{\text{market}}]$$

## Architecture

- Sequence length:  $T = 10$
- Linear projection to  $d_{\text{model}} = 128$
- Learnable positional embeddings
- Transformer Encoder:
  - 2 encoder layers
  - 4 attention heads
  - Feed-forward dimension 256
  - Dropout 0.1

Final representation:

$$h_t = \text{Transformer}(X)_T$$

The embedding  $h_t$  is provided to the actor and critic networks.



# Hybrid MLP–Transformer Architecture

## Motivation

Market observations exhibit temporal structure, while portfolio information represents an instantaneous allocation state.

# Hybrid MLP–Transformer Architecture

## Motivation

Market observations exhibit temporal structure, while portfolio information represents an instantaneous allocation state.

## Market Branch

$$X^{market} = [s_{t-T+1}^{market}, \dots, s_t^{market}]$$

$$h_t^{market} = \text{Transformer}(X^{market})_T$$

# Hybrid MLP–Transformer Architecture

## Motivation

Market observations exhibit temporal structure, while portfolio information represents an instantaneous allocation state.

## Market Branch

$$X^{market} = [s_{t-T+1}^{market}, \dots, s_t^{market}]$$

$$h_t^{market} = \text{Transformer}(X^{market})_T$$

## Portfolio Branch

$$h_t^{portfolio} = \text{MLP}(s_t^{portfolio})$$

## Fusion

$$z_t = [h_t^{market}, h_t^{portfolio}]$$

Concatenation followed by a projection layer before being passed to the actor and critic networks.

# Data Processing

- Rolling window sequences ( $T = 10$ )
- Online feature-wise normalization
- Reward normalization via rolling statistics

## Why it matters

- Handles non-stationary distributions
- Prevents scale dominance

# Training Performance

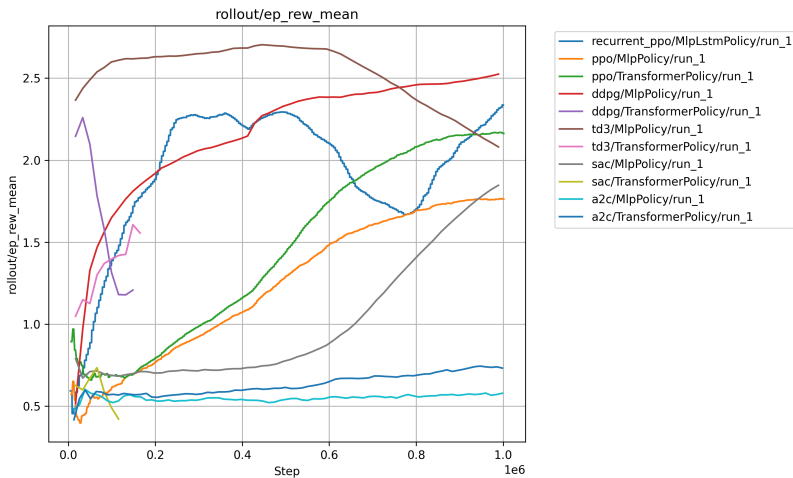


Figure: Training reward evolution

# Financial Performance

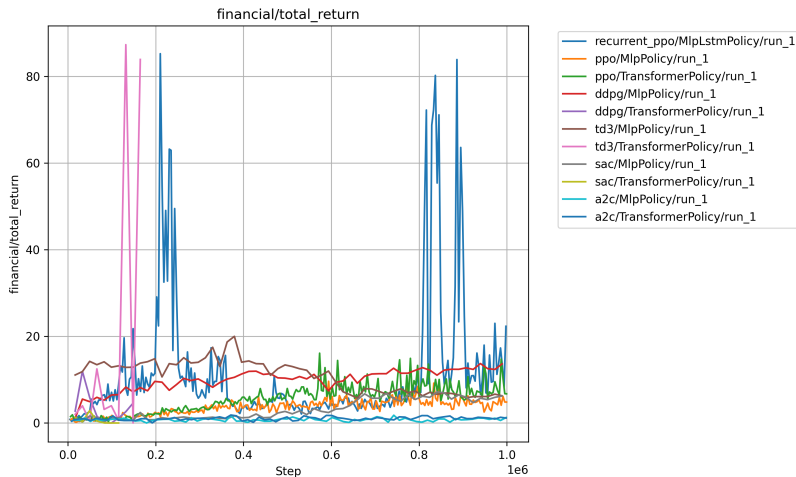


Figure: Total return across models

# Risk-Adjusted Performance

## Key Findings

- PPO: most stable learning behavior
- DDPG (MLP): strongest overall performance
- Transformers: higher representation power but lower stability in off-policy settings
- LSTM: strong performance with unstable long-horizon behavior

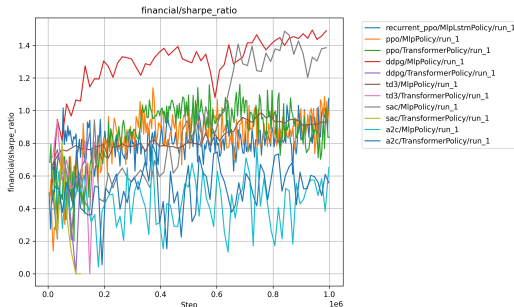


Figure: Sharpe ratio comparison

# Backtesting Evaluation

## Evaluation Setup

- Out-of-sample period:
  - Jan 2026 – Mar 2026
- Dow Jones 30 constituents
- Metrics:
  - Total Return
  - Portfolio Value
  - Sharpe Ratio

## Selected Models

- DDPG (MLP)
- PPO (Transformer)
- PPO (Hybrid)

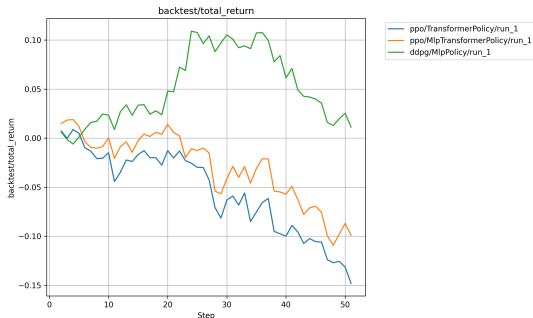


Figure: Out-of-sample total return.



# Portfolio Value & Risk-Adjusted Performance

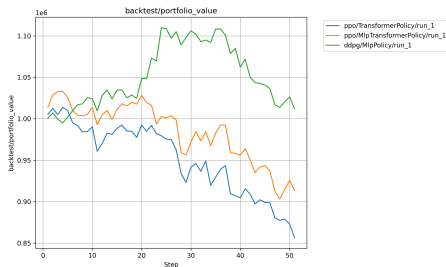


Figure: Portfolio value evolution

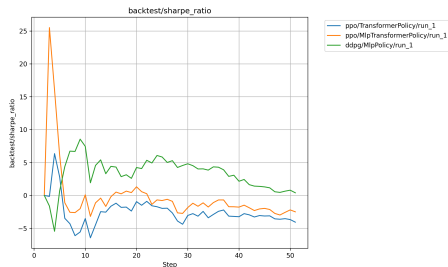


Figure: Sharpe ratio evolution

## Key Observation

DDPG (MLP) maintains the strongest portfolio growth and the most consistent risk-adjusted performance during the out-of-sample period, while both Transformer-based PPO variants exhibit gradual degradation.

# Fuzzy-Enhanced Risk Management

## Motivation

The original environment relied on hard threshold-based controls:

$$a'_t = \min(a_t, 0)$$

which completely blocked buying actions during periods of high market turbulence.

## Limitation

- Abrupt action suppression
- Discontinuous behavior
- Difficult learning dynamics

## Proposed Solution

A Mamdani Fuzzy Inference System composed of:

- 1 Fuzzy Action Smoother
- 2 Fuzzy Reward Shaper

Both components replace hard decisions with smooth, context-dependent risk adjustments.

# Fuzzy Action Smoother

## Goal

Replace the hard turbulence constraint

$$a'_t = \min(a_t, 0)$$

with a smooth risk-aware action adjustment mechanism.

## Inputs

- Raw Action ( $a_i$ ):

{Sell, Hold, Buy}

- Market Turbulence ( $\tau_t$ ):

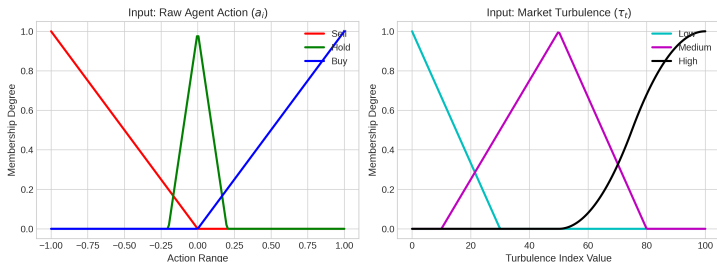
{Low, Medium, High}

## Output

$$a_{t,\text{safe}}$$

Risk-adjusted trading action obtained through Mamdani inference and centroid defuzzification.

# Fuzzy Action Smoother Rules



## Rule Base

- ① IF Turbulence is High **THEN** Action is Sell
- ② IF Turbulence is Medium **AND** Action is Buy **THEN** Action is Hold
- ③ IF Turbulence is Medium **AND** Action is Sell **THEN** Action is Sell
- ④ IF Turbulence is Low **THEN** Action follows the original action

# Fuzzy Reward Shaper

## Goal

Replace the fixed volatility penalty with a context-dependent fuzzy penalty.

Original reward:

$$r_t = \log \left( \frac{V_{t+1} + \epsilon}{V_t + \epsilon} \right) - \lambda e_t v_t$$

## Inputs

- Portfolio Exposure ( $e_t$ ):

{Low, High}

- VIX Volatility ( $v_t$ ):

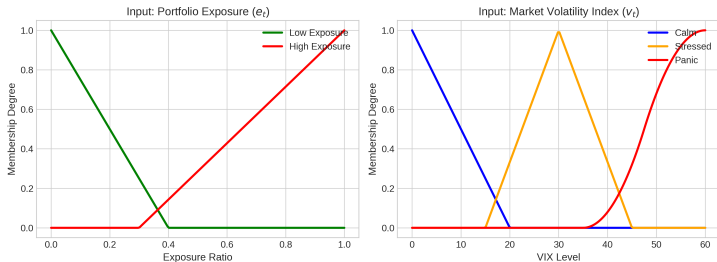
{Calm, Stressed, Panic}

## Output

$$\phi_t \in [0, 1]$$

Adaptive penalty factor.

# Fuzzy Reward Shaper Rules



## Rule Base

- ① IF VIX is Panic **AND** Exposure is High **THEN** Penalty is Severe
- ② IF VIX is Stressed **AND** Exposure is High **THEN** Penalty is Medium
- ③ IF VIX is Calm **OR** Exposure is Low **THEN** Penalty is None

$$r_t = \log \left( \frac{V_{t+1} + \epsilon}{V_t + \epsilon} \right) - \phi_t \lambda_{\text{scale}}$$

# Genetic Algorithm Hyperparameter Optimization

## Motivation

DRL performance is highly sensitive to hyperparameter selection.

Examples include: Learning rate, Batch size, Rollout length etc

## Chromosome Representation

Each hyperparameter configuration is encoded as a chromosome:

$$C = [g_1, g_2, \dots, g_n]$$

where each gene corresponds to a specific hyperparameter.

## Fitness Function

$$Fitness = \text{Final Portfolio Value}$$

obtained after training and evaluating the corresponding DRL agent.

# Genetic Algorithm Evolution Cycle

## Evolutionary Process

- 1 Generate an initial population of random chromosomes
- 2 Train and evaluate each DRL agent
- 3 Compute fitness using the final portfolio value
- 4 Select the highest-performing individuals
- 5 Apply crossover to generate offspring
- 6 Apply mutation to maintain diversity
- 7 Form the next generation and repeat

## Genetic Operators

- **Selection:** favors high-fitness chromosomes
- **Elitism:** preserves the best solution unchanged
- **Uniform Crossover:** combines genes from two parents
- **Mutation:** randomly modifies genes to explore new regions

The cycle is repeated for multiple generations until the best hyperparameter configuration is identified.



# Assignment 2: Fuzzy Irrigation Control for a Greenhouse

## Objective

Regulate greenhouse soil moisture  $m$  toward a desired target  $m^*$  while adapting irrigation to changing temperature conditions.

## Fuzzy Controller

- Inputs:
  - Moisture error  $e = m^* - m$
  - Temperature  $T$

- Output:

$$\Delta v$$

(valve opening adjustment)

- Mamdani inference
- Max aggregation
- Centroid defuzzification

## Controller Improvements

## Evaluation

Three initial moisture levels:

$$m_0 \in \{0.20, 0.35, 0.50\}$$

## Key Results

- Baseline controller converged to target moisture but ignored temperature effects
- Temperature-aware rule base improved adaptation to evaporation changes
- Modified membership functions reduced overshoot
- Smoother valve operation
- Faster convergence for low initial moisture

# Assignment 3: Genetic Algorithms for the TSP

## Core Objective

Minimize cyclic Euclidean distance visiting all cities exactly once:

$$f(\pi) = \sum_{i=1}^N d(\pi_i, \pi_{i+1}), \quad \text{with } \pi_{N+1} = \pi_1$$

## Chromosome & Operators

- **Representation:** (route,  $\sigma$ ) with self-adaptive mutation intensity  $\sigma$ .
- **Crossover:** Linear parent gene combination:  
 $\text{child} = \gamma P_1 + (1 - \gamma) P_2$ .
- **Mutation:** Position swap along route, paired with stochastic updates:  
 $\sigma \leftarrow \sigma + \mathcal{N}(0, \cdot)$

## Selection Strategies & Dynamics

- **Random:** Uniform choice; maximizes genetic diversity but lacks exploitation.
- **Roulette:** Probability proportional to fitness; yields the most geometrically consistent paths.
- **Tournament:** Extracts the best of a random subset; drives aggressive selection pressure and fast convergence.

## Key Takeaway

Selection pressure (Roulette/Tournament) optimizes the exploration-exploitation balance, outperforming random baselines.

# Thank You

Questions?

**Project Repository**

<https://github.com/georrous6/finrl-trading-lab>

Deep Reinforcement Learning Framework for Multi-Asset Trading